Same Accuracy, Different Verdicts: Quantization-Induced Verdict Churn in Small LLM Smart-Contract Auditors

Muneeb Anjum*

Independent Researcher,

Abstract

Small open LLMs are increasingly run as smart-contract auditors in quantized GGUF form, and large models are reached through API aggregators that do not disclose how they are served; both practices are usually justified by aggregate accuracy. We measure *verdict churn*, the share of contracts whose audit verdict changes, on 175 Solidity contracts (143 vulnerable from SmartBugs-curated, 32 patched SWC Registry samples) with a fixed JSON-verdict prompt and a shared parser. Qwen2.5-Coder-0.5B, Llama-3.2-1B and Qwen3-0.6B were run at F16, Q8_0 and Q4_K_M with greedy decoding; an F16 repeat reproduced all outputs byte for byte. At full precision all three flag nearly every contract as vulnerable, so the stated vulnerability class carries the signal, and it churns heavily. For Qwen2.5-Coder, binary accuracy is identical at all precisions while 18.3% (Q8_0) and 45.7% (Q4_K_M) of class verdicts change, with a net loss of correct classes at Q4_K_M (14 vs. 1, McNemar $p = 0.001$) driven by self-contradictory outputs (20.0%→57.1%). Llama's Q4_K_M changes 37.1% of classes while class accuracy slightly improves; its three apparent misses stem from format degradation and vanish under a first-object parser. Churn is model-dependent: Q8_0 changed 1.1–18.3% of classes and no binary verdict, whereas Qwen3 at Q4_K_M lost 15 vulnerable-contract detections ($p = 0.0005$; accuracy 0.811→0.731), partly by ignoring its `/no_think` instruction. Across 21 API endpoints recall is 0.951–1.000 but specificity only 0.400–0.813; router caching makes identical repeats uninformative, whitespace-perturbed repeats bound run-to-run binary churn at 1.2–1.8%, and one model name served by two differently configured providers disagrees on 14 of 172 binary verdicts ($p = 0.013$). Paired verdict agreement should be reported alongside accuracy.

Keywords: smart contracts, Solidity, vulnerability detection, large language models, quantization, GGUF, prediction churn, reproducibility

1. Introduction

Smart contracts are immutable once deployed, so vulnerabilities that slip through review are expensive, and automated analysis has long been part of the auditing workflow [1–4]. Large language models (LLMs) are now used as auditors as well, either alone or combined with program analysis [5–8]. A practical consequence of this trend is that the same auditor is increasingly run in compressed form: small open models are distributed as quantized GGUF files and executed on laptops and CI machines through llama.cpp [9], while proprietary models are reached through aggregators that do not disclose how, or at what precision, they are served.

Quantized models are usually accepted on the strength of an aggregate metric: if accuracy or F1 barely moves, the compressed model is treated as a drop-in replacement [10, 11]. Work on compressed classifiers and LLMs has shown that this inference is unsafe, because a model can keep its accuracy while changing many individual decisions [12–14]; our earlier study found the same pattern in quantized prompt-injection detectors [15]. For an auditor this matters directly: a report that names a different vulnerability class, or silently drops a finding, is a different report, whatever the benchmark score says.

This paper measures *verdict churn*, the share of contracts whose audit verdict changes, when small open LLM auditors are quantized from F16 to Q8_0 and Q4_K_M, and complements it with an external arm that probes run-to-run and cross-provider churn of large proprietary models behind a commercial API router. Our contributions are:

1. A paired, fully logged measurement of quantization-induced churn for small open models (Qwen2.5-Coder-0.5B-Instruct, Llama-3.2-1B-Instruct and Qwen3-0.6B) on 175 Solidity contracts (143 vulnerable from SmartBugs-curated, 32 patched SWC Registry samples), with greedy decoding and an F16 repeat run that establishes a zero noise floor.

*Corresponding author. ORCID: 0009-0001-2042-7656

URL: `https://orcid.org/0009-0001-2042-7656` (Muneeb Anjum)

2. Evidence that churn is often *accuracy-neutral*: for Qwen2.5-Coder-0.5B binary accuracy and F1 are identical at all three precisions, yet the stated vulnerability class changes on 18.3% of contracts at Q8_0 and 45.7% at Q4_K_M, with a significant net loss of correct class verdicts at Q4_K_M (McNemar exact $p = 0.001$). The magnitude is strongly model-dependent: Qwen3-0.6B at Q8_0 changes only 1.1% of class verdicts, while at Q4_K_M it loses 15 detections of vulnerable contracts (16 negative binary flips, $p = 0.0005$).
3. A characterisation of the failure modes behind the churn: self-contradictory verdicts (`vulnerable: true` with class `none`), format degradation (multiple conflicting JSON objects, which makes Llama's apparent "missed vulnerabilities" depend on the parser policy), and loss of control-token compliance (Qwen3 Q4_K_M ignoring `/no_think`), alongside genuine explicit false negatives.
4. An external arm over 22 API endpoints showing high recall but low specificity, a statistically significant cross-provider disagreement for one model name (14 of 172 binary verdicts, $p = 0.013$), and the methodological finding that response caching in the router makes identical temperature-0 repeats uninformative about determinism.

2. Related work

Smart-contract vulnerability detection. Classic tools rely on symbolic execution and static analysis, e.g. Oyente [1] and Slither [3]; attack taxonomies were surveyed by Atzei et al. [2]. Durieux et al. introduced SmartBugs and the curated, annotated vulnerable-contract set used here, and found that the analysed tools disagree widely and that the best detectors are also the noisiest [4, 16]. The DASP Top 10 [17] provides the category scheme of SmartBugs-curated, and the SWC Registry [18] provides weakness descriptions with vulnerable and fixed samples.

LLMs as smart-contract auditors. GPTLens frames LLM auditing as a generate-then-discriminate process to reduce false positives [5]; GPTScan combines GPT with program analysis to find logic vulnerabilities [6]; David et al. evaluate general-purpose LLMs on audited contracts and report many false positives [7]; Chen et al. find that ChatGPT has high recall but limited precision on smart-contract vulnerability detection, and identify uncertainty in its answers and limited code length as robustness problems [8]. These studies use full-precision proprietary models; none measures how per-contract verdicts move when the model is compressed.

Quantization and its hidden effects. Post-training quantization methods such as LLM.int8() [19], GPTQ [20] and AWQ [21], and llama.cpp's k-quant GGUF formats [9], are typically validated by perplexity and benchmark accuracy [10, 11]. Hong et al. show that compression can change trustworthiness properties not captured by accuracy [22], and Egashira et al. show that quantization can even be exploited to make a model behave maliciously only after quantization [23]. Dutta et al. argue that accuracy hides large "flips" between baseline and compressed LLMs [13], echoing earlier work on what compressed networks forget [12] and on prediction churn between model versions [14]. Our previous study measured this effect for prompt-injection detectors [15]; here we move to generative auditors that emit a structured verdict.

Nondeterminism and prompt sensitivity. Temperature-0 decoding is not deterministic in practice for hosted LLMs [24, 25], and LLM outputs are sensitive to semantically irrelevant prompt-format changes [26]. Both effects confound any attempt to attribute churn to quantization, which is why we measure a local noise floor and treat our API repeats as upper bounds.

3. Dataset

Vulnerable contracts. We use all 143 contracts of SmartBugs-curated [27] at commit `230e6491` (full hash in the availability statement), each labelled with its DASP folder; every contract carries a single DASP category in the repository's `vulnerabilities.json`. The class distribution is given in Table 1.

Safe contracts. As negatives we use every `*_fixed.sol` sample embedded in the SWC Registry documentation [18] (`entries/docs/SWC-*.md`) at commit `1b622707`, 32 contracts in total, labelled `none`. These are the registry's own patched versions, and many pair with SmartBugs originals (e.g. integer-overflow, incorrect-constructor-name and guess-the-random-number samples). "Fixed for its SWC" does not certify freedom from every DASP issue, and a few samples target non-DASP weaknesses such as floating pragmas or unused variables; we did not find a larger credible public set of verified-safe contracts.

Table 1: Evaluation set (175 contracts). Vulnerable contracts carry their SmartBugs-curated DASP folder label; safe contracts are the SWC Registry `*_fixed` samples, labelled `none`.

Label	n	Source
`unchecked_low_level_calls`	52	SmartBugs-curated
`none`	32	SWC Registry (fixed)
`reentrancy`	31	SmartBugs-curated
`access_control`	18	SmartBugs-curated
`arithmetic`	15	SmartBugs-curated
`bad_randomness`	8	SmartBugs-curated
`denial_of_service`	6	SmartBugs-curated
`time_manipulation`	5	SmartBugs-curated
`front_running`	4	SmartBugs-curated
`other`	3	SmartBugs-curated
`short_addresses`	1	SmartBugs-curated
Total	175	143 vulnerable, 32 safe

Cleaning and truncation. All comments are stripped, because SmartBugs files carry `// <yes> <report> CLASS` and `@vulnerable_at_lines` annotations that would leak the label; a programmatic check confirmed that no such annotation remains. Runs of eight or more interior spaces are collapsed, because some honeypot contracts hide code far to the right of the visible line. Code longer than 4,000 characters is cut at the last line boundary and marked `// ... [truncated]`; this affects 15 of the 175 contracts and is logged per record. File and contract-folder names are not shown to the model, but identifiers inside the code are kept.

4. Method

Task and prompt. Every model receives the same two-message chat: the system prompt "You are a smart contract security auditor." and a user message that asks for an audit, lists the ten DASP categories plus `none`, and requests only a JSON object `{"vulnerable": true or false, "class": "<one category or none>"}`, followed by the contract in a Solidity code block. For Qwen3 the official `/no_think` soft switch is appended to disable thinking mode.

Local arm. We evaluate Qwen2.5-Coder-0.5B-Instruct [28], Llama-3.2-1B-Instruct [29] and Qwen3-0.6B [30] at F16, Q8_0 and Q4_K_M in GGUF format, run with llama-cpp-python 0.3.36 [31] on two CPU threads with `n_ctx`=4096, temperature 0, `top_k`=1, seed 1234 and a generation budget of 48 tokens (64 for Qwen3, whose output begins with an empty think block). The Qwen2.5-Coder files are the official ones, the Llama files are community imatrix-calibrated quantizations, and the Qwen3 Q8_0 and Q4_K_M files were produced locally from the official F16 file without an importance matrix (Appendix Appendix A). To measure the noise floor, the Qwen2.5-Coder F16 run was repeated in reversed contract order, so that any key-value prefix reuse differs between the two runs.

External API arm. The same prompt and truncation were sent to large models through an OpenAI-compatible commercial aggregator that routes requests to several provider back-ends (identified by the prefixes `agy`, `cap`, `clp` and `opencode-go` in model identifiers). Decoding used temperature 0 and a 4,096-token budget, because reasoning models need room before they emit the verdict; the `/no_think` suffix was not used. Each endpoint was queried once over all contracts (`r1`). For a subset we also (i) repeated identical requests (`r2`), (ii) sent two uncached repeats whose user message ends with one extra whitespace character (`ws1`: newline; `ws2`: space and newline), and (iii) compared the same model name served through two providers. The router does not disclose weights, versions, serving precision or any added system prompt.

Shared verdict parser. Both arms use the same parser. It removes `<think>` blocks, takes the *last* JSON object containing `"vulnerable"` that can be decoded (with a regular-expression fallback on the key), maps `vulnerable: false` to class `none`, and maps `vulnerable: true` with class `none` or an unknown class to `other`. In the local arm an unparsed output counts as "no detection"; in the API arm unparsed responses (refusals, empty outputs) are excluded from accuracy and reported separately. For diagnostics only, a strict first-object parser records whether the first JSON object is self-contradictory (`true` with class `none`), and we count outputs containing more than one verdict object.

Table 2: Local models: aggregate metrics on the 175 contracts (positive class = vulnerable). "Self-contr." is the share of outputs whose first JSON object states `vulnerable: true` with class `none`; "Multi-obj." counts outputs containing more than one verdict object. Class accuracy is exact-match on the 11-way label (10 DASP classes + `none`).

Model	Quant.	Parse fail	Self-contr.	Multi-obj.	Acc.	F1	Recall	Spec.	Class acc. (vuln.)	11-way acc.
Qwen2.5-Coder-0.5B	F16	0.000	0.200	0	0.817	0.899	1.000	0.000	0.133	0.109
Qwen2.5-Coder-0.5B	F16 (repeat)	0.000	0.200	0	0.817	0.899	1.000	0.000	0.133	0.109
Qwen2.5-Coder-0.5B	Q8_0	0.000	0.274	0	0.817	0.899	1.000	0.000	0.105	0.086
Qwen2.5-Coder-0.5B	Q4_K_M	0.000	0.571	0	0.817	0.899	1.000	0.000	0.042	0.034
Llama-3.2-1B	F16	0.000	0.000	0	0.817	0.899	1.000	0.000	0.210	0.171
Llama-3.2-1B	Q8_0	0.000	0.000	1	0.817	0.899	1.000	0.000	0.189	0.154
Llama-3.2-1B	Q4_K_M	0.000	0.000	10	0.800	0.889	0.979	0.000	0.266	0.217
Qwen3-0.6B	F16	0.000	0.006	0	0.811	0.896	0.993	0.000	0.126	0.103
Qwen3-0.6B	Q8_0	0.000	0.006	0	0.811	0.896	0.993	0.000	0.126	0.103
Qwen3-0.6B	Q4_K_M	0.040	0.000	0	0.731	0.844	0.888	0.031	0.119	0.103

Table 3: Local models: churn against the same model at F16 ($n = 175$ paired contracts). Binary flips change the vulnerable/safe verdict; 11-way flips change the stated label. Neg./pos. = vulnerable→safe / safe→vulnerable. McNemar b/c = reference correct & quantized wrong / the reverse; exact two-sided binomial p. Intervals are 95% Wilson. "Text changed" counts byte-level differences in the raw output.

		Binary verdict				11-way verdict					
Model	Quant.	Flips (95% CI)	Neg./pos.	b/c, p	ΔAcc.	Flips	Rate (95% CI)	McNemar b/c	p	Δ11-way acc.	Text changed
Qwen2.5-Coder-0.5B	F16 (repeat)	0 [0.000,0.021]	0/0	0/0, 1.000	0.000	0	0.000 [0.000,0.021]	0/0	1.000	0.000	0
Qwen2.5-Coder-0.5B	Q8_0	0 [0.000,0.021]	0/0	0/0, 1.000	0.000	32	0.183 [0.133,0.247]	4/0	0.125	-0.023	48
Qwen2.5-Coder-0.5B	Q4_K_M	0 [0.000,0.021]	0/0	0/0, 1.000	0.000	80	0.457 [0.385,0.531]	14/1	0.001	-0.074	137
Llama-3.2-1B	Q8_0	0 [0.000,0.021]	0/0	0/0, 1.000	0.000	13	0.074 [0.044,0.123]	3/0	0.250	-0.017	31
Llama-3.2-1B	Q4_K_M	3 [0.006,0.049]	3/0	3/0, 0.250	-0.017	65	0.371 [0.303,0.445]	3/11	0.057	+0.046	92
Qwen3-0.6B	Q8_0	0 [0.000,0.021]	0/0	0/0, 1.000	0.000	2	0.011 [0.003,0.041]	0/0	1.000	0.000	24
Qwen3-0.6B	Q4_K_M	16 [0.057,0.143]	16/0	15/1, <0.001	-0.080	28	0.160 [0.113,0.222]	1/1	1.000	0.000	65

Metrics. Binary metrics treat "vulnerable" as the positive class; specificity is measured on the 32 safe contracts. Class accuracy is exact match against the 11-way label. For each quantized run we pair every contract with the same model's F16 output and count binary flips (split into negative flips, vulnerable→safe, and positive flips), 11-way verdict flips, and byte-level changes of the raw text. Flip rates carry 95% Wilson intervals [32]. Whether quantization changes correctness is tested with an exact two-sided McNemar test [33] on the discordant pairs (b: reference correct and quantized wrong; c: the reverse), both for the binary and for the 11-way verdict.

5. Local results

The binary task is degenerate for these models. At full precision Qwen2.5-Coder-0.5B and Llama-3.2-1B label every contract as vulnerable, and Qwen3-0.6B all but one, including every fixed contract (specificity 0, Table 2). Binary accuracy therefore sits at or just below the base rate of $143/175 = 0.817$, and for the first two models binary churn is near zero by construction. The informative signal is the class verdict.

Zero noise floor. The reversed-order F16 repeat of Qwen2.5-Coder-0.5B reproduced all 175 outputs byte for byte (0 text changes, 0 flips; Table 3). With greedy decoding on this stack, every difference reported below can be attributed to the change of weights rather than to run-to-run nondeterminism.

Accuracy-neutral class churn. For Qwen2.5-Coder-0.5B, binary accuracy, F1 and recall are identical at F16, Q8_0 and Q4_K_M, and there is not a single binary flip. Yet the 11-way verdict changes on 32 contracts (18.3%, 95% CI 13.3–24.7%) at Q8_0 and on 80 contracts (45.7%, 38.5–53.1%) at Q4_K_M. At Q4_K_M the changes are not neutral: 14 contracts lose a correct class and only 1 gains one (McNemar $p = 0.001$), and class accuracy on vulnerable contracts falls from 0.133 to 0.042. At Q8_0 the direction is the same (4/0) but not significant ($p = 0.125$).

Failure mode: self-contradictory verdicts. The Qwen2.5-Coder degradation has a recognisable form. The share of outputs that state `vulnerable: true` together with class `none` rises from 20.0% at F16 to 27.4% at Q8_0 and 57.1% at Q4_K_M (Table 2). The shared parser maps these to `other`, so they keep the binary verdict intact while destroying the class; an evaluation that reports only binary accuracy cannot see them.

Churn and accuracy change are decoupled. For Llama-3.2-1B, Q4_K_M changes the class verdict on 65 contracts (37.1%, 30.3–44.5%), but these changes are net-neutral to slightly positive for correctness: 3 correct→wrong against 11 wrong→correct ($p = 0.057$), and 11-way accuracy rises by 4.6 points. A practitioner comparing accuracies would conclude that Q4_K_M is at least as good as F16, while more than a third of the reports it produces differ from the full-precision ones.

Table 4: 11-way verdict flips by ground-truth label (flipped / n), each quantized model against its own F16 reference.

True label	n	Qwen2.5-Coder-0.5B Q8_0	Qwen2.5-Coder-0.5B Q4_K_M	Llama-3.2-1B Q8_0	Llama-3.2-1B Q4_K_M	Qwen3-0.6B Q8_0	Qwen3-0.6B Q4_K_M
`unchecked_low_level_calls`	52	13	30	3	25 (2 neg.)	1	7 (6 neg.)
`none`	32	2	15	2	10	0	7 (1 neg.)
`reentrancy`	31	4	7	3	7	0	3 (3 neg.)
`access_control`	18	4	11	0	2	0	1 (1 neg.)
`arithmetic`	15	3	5	3	8	1	5 (1 neg.)
`bad_randomness`	8	1	5	1	5 (1 neg.)	0	1 (1 neg.)
`denial_of_service`	6	2	1	0	1	0	1
`time_manipulation`	5	1	2	0	3	0	1 (1 neg.)
`front_running`	4	1	2	1	2	0	0
`other`	3	0	1	0	2	0	1 (1 neg.)
`short_addresses`	1	1	1	0	0	0	1 (1 neg.)

Missed vulnerabilities are format degradation. Among the Qwen2.5-Coder and Llama configurations, Llama-3.2-1B at Q4_K_M is the only one with binary flips: 3 negative flips (1.7%, 0.6–4.9%), all on truly vulnerable contracts (McNemar 3/0, $p = 0.25$). All three occur in outputs that contain several conflicting JSON objects, a pattern seen in 10 of 175 Q4_K_M outputs and in none at F16. In each case the first object says `vulnerable: true` and a later one says `false`; the shared last-object parser therefore records a miss, whereas a first-object parser records no binary flip at all. Because the 48-token budget also truncates these rambling outputs, the position of the "last" object is partly an artefact of the budget. We therefore report these three cases as quantization-induced *format degradation* whose binary interpretation depends on the parser policy, not as clean semantic misses.

Qwen3-0.6B: little churn at Q8_0, real misses at Q4_K_M. Qwen3-0.6B behaves differently from the other two models. At F16 it also flags almost everything (174 of 175 contracts, specificity 0) and answers `access_control` for 160 of them. Q8_0 is nearly churn-free: no binary flip and only 2 class flips (1.1%, 0.3–4.1%), although 24 raw outputs differ in formatting. Q4_K_M, in contrast, produces the only statistically significant binary degradation in the local arm: 16 negative flips (9.1%, 5.7–14.3%) and no positive flip, of which 15 are truly vulnerable contracts and one is a fixed contract (now correctly called safe). McNemar gives 15/1, $p = 0.0005$, and binary accuracy falls from 0.811 to 0.731. The 16 flips have two distinct causes. In 7 cases the Q4_K_M model ignores the `/no_think` switch and starts a free-text reasoning trace, which the 64-token budget cuts off before any verdict; these outputs are unparsed and therefore count as "no detection". In the other 9 cases the model emits a well-formed `{"vulnerable": false, "class": "none"}` (once inside an unclosed think block), a genuine semantic miss on 8 vulnerable contracts plus the one corrected fixed contract. No F16 or Q8_0 output of Qwen3 deviates from the empty-think-block format, whereas 8 Q4_K_M outputs do. Unlike the Llama case, a first-object parser gives the same 16 flips, so this result is not parser-sensitive; but if unparsed outputs were excluded instead of counted as misses, 9 binary flips would remain. The 11-way verdict changes on 28 contracts (16.0%, 11.3–22.2%), with no net change in class correctness (1/1). For Qwen3, therefore, aggressive quantization is not accuracy-neutral: it degrades control-token compliance and recall, and the degradation is visible in aggregate accuracy.

Llama-3.2-1B at Q8_0. At Q8_0, Llama-3.2-1B keeps every binary verdict (0 flips) and changes the class on 13 contracts (7.4%, 4.4–12.3%); the discordant pairs point towards a small loss (3 correct→wrong, 0 the reverse, $p = 0.25$; 11-way accuracy −1.7 points). One Q8_0 output contains more than one verdict object, against 10 at Q4_K_M and none at F16, so format degradation grows with the compression level. Across the three models, Q8_0 class churn therefore ranges from 1.1% (Qwen3) through 7.4% (Llama) to 18.3% (Qwen2.5-Coder), and Q4_K_M from 16.0% to 45.7%.

Where churn concentrates. Table 4 breaks the 11-way flips down by ground-truth label. Unchecked low-level calls, the largest class, accounts for the most flips (or ties for the most) in every configuration with more than two flips, and the fixed (`none`) contracts also churn substantially at Q4_K_M (7–15 of 32) even though, except for one Qwen3 contract, their binary verdict never changes. Per-class counts are small, so we do not test them individually.

6. External API arm

High recall, low specificity. Twenty-two endpoints returned usable responses; 21 of them returned at least 65 parsed verdicts and one (`clp/lfm-2.5-2.6b-free`) only 20, which we exclude from summary ranges. Across the 21, recall on parsed responses is 0.951–1.000, but specificity on the safe contracts is only 0.400–0.813 (Table 5): large models, like the small ones, flag most patched contracts as vulnerable. Parse failures are almost all refusals or empty outputs rather than malformed JSON: three Gemini endpoints refused 44 to 110 of the 175 contracts,

Table 5: External API arm: first run (`r1`) of each endpoint, temperature 0. Metrics are computed on parsed responses only; refusals and empty outputs are counted as parse failures and reported separately. Specificity is over the parsed safe contracts. Endpoint names are the router's provider prefix and model identifier, as returned; backends, weights and serving precision are not disclosed by the router. †Only 20 responses, excluded from summary ranges.

Endpoint	Covered	Parsed	Refusals	Empty	Acc.	F1	Recall	Spec.	Class acc. (vuln.)
`agy/claude-opus-4-6-thinking`	175	175	0	0	0.874	0.927	0.979	0.406	0.748
`agy/claude-sonnet-4-6`	175	175	0	0	0.903	0.944	0.993	0.500	0.818
`agy/gemini-3-flash`	175	131	44	0	0.924	0.955	1.000	0.615	0.781
`agy/gemini-3.1-flash-lite`	175	175	0	0	0.891	0.936	0.972	0.531	0.636
`agy/gemini-3.1-pro-high`	175	65	110	0	0.923	0.950	0.959	0.812	0.837
`agy/gemini-3.5-flash-lite`	175	175	0	0	0.914	0.950	0.986	0.594	0.755
`agy/gemini-3.6-flash-tiered`	175	85	90	0	0.953	0.973	1.000	0.714	0.859
`agy/gemini-3.7-flash-tiered`	175	175	0	0	0.903	0.943	0.979	0.562	0.755
`agy/gemini-3.8-flash-tiered`	175	153	0	0	0.941	0.966	0.992	0.680	0.844
`agy/gpt-oss-120b-medium`	175	175	0	0	0.863	0.919	0.951	0.469	0.685
`cap/claude-opus-4.6`	172	172	0	0	0.936	0.962	0.993	0.677	0.716
`cap/deepseek-v4-flash-0731`	175	175	0	0	0.926	0.956	0.993	0.625	0.706
`cap/glm-5.2`	175	175	0	0	0.926	0.956	0.993	0.625	0.713
`cap/qwen3.8-27b`	175	175	0	0	0.926	0.956	0.986	0.656	0.692
`clp/cline-free_mimo-v2.6-flash`	175	175	0	0	0.891	0.936	0.965	0.562	0.664
`clp/cline-free_muse-spark-1.3`	175	175	0	0	0.920	0.952	0.979	0.656	0.678
`clp/dots-3-note-preview-free`	145	145	0	0	0.952	0.975	0.958	0.667	0.662
`clp/laguna-s-2.1-free`	175	175	0	0	0.886	0.934	0.986	0.438	0.608
`clp/lfm-2.5-2.6b-free`†	20	20	0	0	0.650	0.788	0.650	–	0.500
`clp/nemotron-3-nano-omni-30b-free`	175	167	0	0	0.856	0.916	0.956	0.400	0.540
`clp/nemotron-3-super-120b-free`	175	175	0	0	0.880	0.930	0.979	0.438	0.615
`opencode-go/deepseek-v4.1-flash`	175	117	0	58	0.949	0.971	0.990	0.706	0.780

Table 6: External API arm: paired churn. (A) Same model name served by two providers. (B_{cache}) Identical repeated request; the router returned cached responses, so these zeros carry no information about determinism. (B_{ws}) Two uncached runs whose prompts differ by one trailing whitespace character (`ws1` = "\n", `ws2` = " \n"); an upper bound on run-to-run churn. Flips are counted on contracts parsed in both runs; parse-status flips are contracts parsed in one run only. McNemar b/c = first run correct & second wrong / the reverse.

Type	Pair	Both parsed	Parse-status flips	Binary flips (%)	11-way flips (%)	Text differs	b/c	p
A	claude-opus-4.6: cap vs agy	172	0	14 (8.1)	31 (18.0)	31	12/2	0.013
B_{cache}	agy/claude-sonnet-4-6 r1 vs r2	175	0	0 (0.0)	0 (0.0)	0	0/0	1.000
B_{cache}	agy/claude-opus-4-6-thinking r1 vs r2	175	0	0 (0.0)	0 (0.0)	0	0/0	1.000
B_{cache}	opencode-go/deepseek-v4.1-flash r1 vs r2	117	0	0 (0.0)	0 (0.0)	0	0/0	1.000
B_{cache}	cap/qwen3.8-27b r1 vs r2	175	0	0 (0.0)	0 (0.0)	0	0/0	1.000
B_{ws}	agy/claude-sonnet-4-6 ws1 vs ws2	175	0	3 (1.7)	9 (5.1)	11	2/1	1.000
B_{ws}	agy/claude-opus-4-6-thinking ws1 vs ws2	175	0	3 (1.7)	5 (2.9)	5	1/2	1.000
B_{ws}	opencode-go/deepseek-v4.1-flash ws1 vs ws2	109	22	2 (1.8)	3 (2.8)	25	2/0	0.500
B_{ws}	clp/laguna-s-2.1-free ws1 vs ws2	84	0	1 (1.2)	11 (13.1)	11	0/1	1.000

and `opencode-go/deepseek-v4.1-flash` returned 58 empty answers because its hidden reasoning consumed the whole token budget. Since accuracy is computed on parsed items only, endpoints with many refusals are not directly comparable with the others.

Identical repeats are uninformative. The router caches responses. An identical request (same model and messages) returned the same response identifier within about a second, even for reasoning models that needed tens of seconds the first time, and the `user`, `seed`, `top_p`, `n`, `stream` and `max_tokens` fields did not change the cache key. All four identical repeats therefore show zero churn and byte-identical text (B_{cache} in Table 6). These zeros say nothing about determinism; a study that re-ran a temperature-0 audit through such a router would wrongly conclude that the auditor is perfectly stable.

Uncached repeats as upper bounds. To bypass the cache we appended a single whitespace character to the user message, producing two uncached runs (`ws1`, `ws2`). Between them, binary verdicts changed on 1.7% of contracts for `agy/claude-sonnet-4-6` and `agy/claude-opus-4-6-thinking`, 1.8% of both-parsed contracts for `opencode-go/deepseek-v4.1-flash` (with 22 further contracts parsed in only one run) and 1.2% for `clp/laguna-s-2.1-free`; class verdicts changed on 2.9–5.1% for the first three and 13.1% for laguna. No McNemar test is significant ($p \geq 0.5$). Because the two prompts differ by one character, these figures mix sampling nondeterminism with prompt sensitivity [26] and should be read as upper bounds on run-to-run churn.

Same model name, different provider. The two cross-provider pairs we planned (`qwen3.8-27b` and `deepseek-v4.1-flash` through a second, free provider) produced no successful responses during the session because of persistent rate limiting, so cross-provider churn rests on one substitute pair: `claude-opus-4.6` through `cap` and through `agy`, where the `agy` identifier carries a `-thinking` suffix. On the 172 contracts parsed by both, the binary verdict differs on 14 (8.1%) and the class on 31 (18.0%); the discordant pairs favour `cap` (12 correct-only-on-`cap` against 2, exact $p = 0.013$), and 10 of the 14 binary flips are safe contracts. This is larger than the whitespace upper bound measured within one provider. However, the two endpoints are visibly configured differently, with mean completion lengths of 1,483.1 tokens on `cap` and 17.1 on `agy`; the comparison

is therefore "same model name, different serving stack", not a controlled same-weights experiment, and it cannot isolate serving precision.

7. Discussion

Across both arms the common thread is that aggregate scores are a poor proxy for per-contract stability. Locally, quantization left binary accuracy unchanged for Qwen2.5-Coder-0.5B while up to 45.7% of its class verdicts changed, and Llama-3.2-1B's class verdicts churned by 37.1% while its class accuracy slightly improved. Qwen3-0.6B shows the opposite risk: almost no churn at Q8_0, but at Q4_K_M a loss of 15 vulnerable-contract detections that *is* visible in accuracy, partly because the quantized model stopped honouring the `/no_think` control instruction. Churn at a given bit width is therefore model-specific and cannot be predicted from one family; Q8_0 ranged from 1.1% (Qwen3) through 7.4% (Llama) to 18.3% (Qwen2.5-Coder) class churn on the same contracts, and no Q8_0 configuration changed a single binary verdict. Behind the router, the same model name delivered through two providers disagreed on 8.1% of binary verdicts.

For an auditing pipeline the practical implications are: (i) accept a quantized auditor on paired agreement with its full-precision reference, per class and per verdict, not on accuracy alone; (ii) monitor output-format health (self-contradictions, multiple verdict objects, unexpected reasoning traces), because format and control-token degradation is the first visible symptom of aggressive quantization in small models and it interacts with the parser and the token budget; (iii) when using hosted models, pin the provider, record response identifiers, and never use identical repeats to claim stability when a cache may sit in the path. At the same time, the small models studied here are not usable auditors (specificity 0 at full precision), so our local results speak to the stability of compressed auditors, not to their usefulness.

8. Limitations

- **Model scale and families.** The local arm covers three small models (0.5–1B parameters) from two families. None discriminates vulnerable from patched contracts, so binary churn is near zero by construction and the evidence rests on the class verdict. Larger open models (1.5–4B and above), which need a GPU, are required to measure binary churn on a model that actually separates the classes; a few-shot prompt may also help.
- **Mixed quantization sources.** Official (Qwen2.5-Coder), community imatrix (Llama) and local non-imatrix (Qwen3) quantizations differ in calibration, so cross-family comparisons of churn magnitude are confounded by the quantization recipe; in particular, the Qwen3 Q4_K_M degradation may partly reflect the absence of importance-matrix calibration.
- **Small and imperfect safe set.** Only 32 safe contracts, each "fixed for one SWC" rather than certified free of all DASP issues; specificity estimates are coarse. Many SmartBugs contracts use old Solidity (0.4.x).
- **Truncation and token budget.** 15 of 175 contracts are truncated at 4,000 characters. Locally, the 48-token budget (64 for Qwen3) truncates verbose answers and shapes which JSON object comes last; the 7 Qwen3 Q4_K_M outputs that began a reasoning trace were cut before any verdict, and a larger budget might have produced one; in the API arm the budget is 4,096 tokens, and one endpoint exhausted it on hidden reasoning.
- **Single prompt and seed.** Greedy decoding was shown to be deterministic on one model (Qwen2.5-Coder F16) only. Contract identifiers left in the code (e.g. names that echo the vulnerability) can hint at the class, and well-known SmartBugs contracts may be in the models' training data.
- **Parser choices.** Mapping `true+none` and unknown classes to `other` merges them with the genuine `other` class (3 contracts). The local arm counts parse failures as "no detection", while the API arm excludes them; accuracies are therefore not directly comparable across arms.
- **Closed router.** Backends, model versions, quantization and serving precision behind the API router are unknown, and reported prompt-token counts differ by provider (412.5–464.9 per-endpoint mean on `clp`, 536.6–594.0 on `agy`, 682.1–696.1 on `cap`), so a hidden system prompt cannot be excluded. Response caching prevents identical repeats from measuring nondeterminism, and the whitespace workaround mixes nondeterminism with prompt sensitivity.
- **Single cross-provider pair.** The planned cross-provider pairs failed because of rate limiting; the substitute pair uses differently configured endpoints (completion lengths of 1,483.1 vs 17.1 tokens), so it does not isolate precision or weights.

- **Coverage gaps and drift in the API arm.** Several endpoints have partial coverage (Table 5), and `clp/laguna-s-2.1-free` changed behaviour during the session (mean completion length 26.9 tokens in `r1` against about 1,000 in the later whitespace runs), suggesting a back-end switch; comparisons involving its `r1` run should be read with care. Accuracy and F1 are computed on parsed items only.

9. Conclusion

Quantizing small LLM smart-contract auditors from F16 to Q8_0 and Q4_K_M changed between 1.1% and 45.7% of their class verdicts, against a deterministic F16 baseline that rules out run-to-run noise as the cause. For Qwen2.5-Coder-0.5B and Llama-3.2-1B the churn was largely invisible to binary accuracy; its mechanisms were self-contradictory verdicts and format degradation, and the few apparent "missed vulnerabilities" depended on how the output was parsed. For Qwen3-0.6B, Q8_0 was nearly lossless but Q4_K_M dropped 15 vulnerable-contract detections, partly by ignoring its thinking-mode control instruction. Behind a commercial router, large models showed high recall but low specificity, identical repeats were masked by caching, and one model name served through two providers disagreed on 8.1% of binary verdicts. Auditing pipelines that swap model precision or provider should report paired verdict agreement and output-format health alongside accuracy, and should validate each model and bit width separately.

Data and code availability

All contracts come from public repositories at pinned commits (SmartBugs-curated `230e649123477eff332742a59a1c7cc6dc286cab`; SWC Registry `1b6227074ecd180b374e4844153afdda0332f979`). All local models are public GGUF files; their SHA-256 hashes are listed in Appendix A. The dataset builder, the resume-safe runners, the shared verdict parser, the analysis scripts, every raw model output (one JSON line per contract and run, including the raw text) and the table generator used for this manuscript are publicly available at `https://github.com/horizonbymuneeb/smart-contract-verdict-churn` (release v1, MIT licence) and archived together with this preprint on Zenodo (`https://doi.org/10.5281/zenodo.23241640`). The external API arm used a closed commercial aggregator whose backends cannot be pinned; its raw responses, including returned model names, finish reasons, latencies and token counts, are released with the same package, but the calls themselves cannot be guaranteed to be reproducible.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used AI tools to assist with running experiments and drafting the manuscript. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the publication.

CRediT authorship contribution statement

Muneeb Anjum: Conceptualization, Methodology, Software, Investigation, Formal analysis, Data curation, Writing – original draft, Writing – review & editing.

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

References

[1] L. Luu, D.-H. Chu, H. Olickel, P. Saxena, A. Hobor, Making smart contracts smarter, in: Proc. ACM SIGSAC Conference on Computer and Communications Security (CCS), 2016, pp. 254–269. doi:10.1145/2976749.2978309.

[2] N. Atzei, M. Bartoletti, T. Cimoli, A survey of attacks on Ethereum smart contracts (SoK), in: Principles of Security and Trust (POST), Vol. 10204 of LNCS, Springer, 2017, pp. 164–186. doi:10.1007/978-3-662-54455-6_8.

[3] J. Feist, G. Grieco, A. Groce, Slither: A static analysis framework for smart contracts, in: Proc. IEEE/ACM 2nd International Workshop on Emerging Trends in Software Engineering for Blockchain (WETSEB), 2019, pp. 8–15. doi:10.1109/WETSEB.2019.00008.

[4] T. Durieux, J. F. Ferreira, R. Abreu, P. Cruz, Empirical review of automated analysis tools on 47,587 Ethereum smart contracts, in: Proc. ACM/IEEE 42nd International Conference on Software Engineering (ICSE), 2020, pp. 530–541. doi:10.1145/3377811.3380364.

[5] S. Hu, T. Huang, F. İlhan, S. F. Tekin, L. Liu, Large language model-powered smart contract vulnerability detection: New perspectives, in: Proc. 5th IEEE International Conference on Trust, Privacy and Security in Intelligent Systems, and Applications (TPS-ISA), 2023, arXiv:2310.01152.

[6] Y. Sun, D. Wu, Y. Xue, H. Liu, H. Wang, Z. Xu, X. Xie, Y. Liu, GPTScan: Detecting logic vulnerabilities in smart contracts by combining GPT with program analysis, in: Proc. IEEE/ACM 46th International Conference on Software Engineering (ICSE), 2024, arXiv:2308.03314.

[7] I. David, L. Zhou, K. Qin, D. Song, L. Cavallaro, A. Gervais, Do you still need a manual smart contract audit?, arXiv preprint arXiv:2306.12338 (2023).

[8] C. Chen, J. Su, J. Chen, Y. Wang, T. Bi, J. Yu, Y. Wang, X. Lin, T. Chen, Z. Zheng, When ChatGPT meets smart contract vulnerability detection: How far are we?, ACM Transactions on Software Engineering and Methodology 34 (4) (2025) 100, arXiv:2309.05520.

[9] G. Gerganov, ggml-org contributors, llama.cpp: LLM inference in C/C++, https://github.com/ggml-org/llama.cpp (2023).

[10] S. Li, X. Ning, L. Wang, T. Liu, X. Shi, S. Yan, G. Dai, H. Yang, Y. Wang, Evaluating quantized large language models, in: Proc. 41st International Conference on Machine Learning (ICML), Vol. 235 of PMLR, 2024, pp. 28480–28524.

[11] R. Jin, J. Du, W. Huang, W. Liu, J. Luan, B. Wang, D. Xiong, A comprehensive evaluation of quantization strategies for large language models, in: Findings of the Association for Computational Linguistics: ACL 2024, 2024, arXiv:2402.16775.

[12] S. Hooker, A. Courville, G. Clark, Y. Dauphin, A. Frome, What do compressed deep neural networks forget?, arXiv preprint arXiv:1911.05248 (2019).

[13] A. Dutta, S. Krishnan, N. Kwatra, R. Ramjee, Accuracy is not all you need, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.

[14] M. M. Fard, Q. Cormier, K. Canini, M. Gupta, Launch and iterate: Reducing prediction churn, in: Advances in Neural Information Processing Systems (NeurIPS), 2016.

[15] M. Anjum, Quantized guardians: Accuracy-neutral decision churn in compressed prompt-injection detectors, Zenodo preprint (2026). doi:10.5281/zenodo.23233203.

[16] J. F. Ferreira, P. Cruz, T. Durieux, R. Abreu, SmartBugs: A framework to analyze Solidity smart contracts, in: Proc. 35th IEEE/ACM International Conference on Automated Software Engineering (ASE), Tool Demonstrations, 2020, pp. 1349–1352. doi:10.1145/3324884.3415298.

[17] NCC Group, DASP – decentralized application security project top 10 of 2018, https://dasp.co (2018).

[18] SmartContractSecurity, Smart contract weakness classification (SWC) registry, https://github.com/SmartContractSecurity/SWC-registry, commit 1b6227074ecd180b374e4844153afdda0332f979 (2024).

[19] T. Dettmers, M. Lewis, Y. Belkada, L. Zettlemoyer, LLM.int8(): 8-bit matrix multiplication for transformers at scale, in: Advances in Neural Information Processing Systems (NeurIPS), 2022.

[20] E. Frantar, S. Ashkboos, T. Hoefler, D. Alistarh, GPTQ: Accurate post-training quantization for generative pre-trained transformers, in: Proc. ICLR, 2023.

[21] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, S. Han, AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration, in: Proc. Machine Learning and Systems (MLSys), 2024.

[22] J. Hong, et al., Decoding compressed trust: Scrutinizing the trustworthiness of efficient LLMs under compression, in: Proc. ICML, 2024.

[23] K. Egashira, M. Vero, R. Staab, J. He, M. Vechev, Exploiting LLM quantization, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.

[24] S. Ouyang, J. M. Zhang, M. Harman, M. Wang, An empirical study of the non-determinism of ChatGPT in code generation, arXiv preprint arXiv:2308.02828 (2023).

[25] B. Atil, S. Aykent, A. Chittams, L. Fu, R. J. Passonneau, E. Radcliffe, G. R. Rajagopal, A. Sloan, et al., Non-determinism of "deterministic" LLM settings, arXiv preprint arXiv:2408.04667 (2024).

[26] M. Sclar, Y. Choi, Y. Tsvetkov, A. Suhr, Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting, in: Proc. ICLR, 2024.

[27] SmartBugs contributors, SmartBugs-curated: annotated vulnerable Solidity smart contracts, `https://github.com/smartbugs/smartbugs-curated`, commit 230e649123477eff332742a59a1c7cc6dc286cab (2024).

[28] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, et al., Qwen2.5-Coder technical report, arXiv preprint arXiv:2409.12186 (2024).

[29] A. Grattafiori, A. Dubey, et al., The Llama 3 herd of models, arXiv preprint arXiv:2407.21783 (2024).

[30] A. Yang, A. Li, B. Yang, B. Zhang, et al., Qwen3 technical report, arXiv preprint arXiv:2505.09388 (2025).

[31] A. Betlen, contributors, llama-cpp-python: Python bindings for llama.cpp, `https://github.com/abetlen/llama-cpp-python`, version 0.3.36 (2023).

[32] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (158) (1927) 209–212.

[33] Q. McNemar, Note on the sampling error of the difference between correlated proportions or percentages, Psychometrika 12 (2) (1947) 153–157.

Appendix A. Model files

Table A.7: GGUF files used in the local arm (SHA-256). Sources: `Qwen/Qwen2.5-Coder-0.5B-Instruct-GGUF` (official), `bartowski/Llama-3.2-1B-Instruct-GGUF` (community, imatrix-calibrated quants), `ggml-org/Qwen3-0.6B-GGUF` (F16; Q8_0 and Q4_K_M quantized locally with `llama_model_quantize`, no imatrix). The hash of `qwen2.5-coder-0.5b-instruct-q8_0.gguf` was not recorded before the file was deleted.

File	SHA-256
`Llama-3.2-1B-Instruct-f16.gguf`	`1f33ad43d2b85b908ff06fe7002b69806a57359b9b2617ca27d7bdea428ae146`
`Llama-3.2-1B-Instruct-Q4_K_M.gguf`	`6f85a640a97cf2bf5b8e764087b1e83da0fdb51d7c9fab7d0fece9385611df83`
`Llama-3.2-1B-Instruct-Q8_0.gguf`	`432f310a77f4650a88d0fd59ecdd7cebed8d684bafea53cbff0473542964f0c3`
`qwen2.5-coder-0.5b-instruct-fp16.gguf`	`bb69b303c65113199b19c72f9c75bca449200b6cdf807e0f7d8822f669c02e64`
`qwen2.5-coder-0.5b-instruct-q4_k_m.gguf`	`1d9614638d18024d0fbb36575a15f1302a3adf044df10345688ec4f6e1c4ff32`
`Qwen3-0.6B-f16.gguf`	`ab9004daf660cd6a6ba1c07556e74fcceb2b756063ccce3f9c69d3a637b361cc`
`Qwen3-0.6B-Q4_K_M.gguf`	`ebfd42b196c529c3b692c549585e62cc6e817b5a5aff8375b3724cdfbafa651a`
`Qwen3-0.6B-Q8_0.gguf`	`84c0dbe606526d5907251d88ea88b41457f46ce456e9a333d5d2b6245a95cafe`
`qwen2.5-coder-0.5b-instruct-q8_0.gguf`	not recorded